

MANUEL D'UTILISATION

Gunther.ino

« Mode classique »

2026-2027

TABLE DES MATIÈRES

L'ASSOCIATION ET LES GRANDS ACTEURS DU PROJET 3
PRÉSENTATION 4
VUE D'ENSEMBLE 5
ALIMENTATION 6
INSTALLATION ET OUTILS 7
LES BORNIERS 8
COMPOSANTS DE LA CARTE 10
CAPACITÉS DES BROCHES 11
ENTRÉES, SORTIES ET INTERRUPTIONS 12
ENTRÉES ET SORTIES ANALOGIQUES 13
TEMPS ET SON 14
LIAISONS SÉRIE 15
BUS I2C (WIRE) 16
BUS SPI 17
CARTE SD 18
SERVOS 19
EEPROM ET WATCHDOG 20
FONCTIONS PROPRES À LA CARTE 21
EXEMPLES FOURNIS 22
CONTACT ET VERSION 23



L'ASSOCIATION ET LES GRANDS ACTEURS DU PROJET

Créée en 1986, l'Association Aérospatiale Air ESIEA est engagée depuis près de 40 ans dans les événements organisés par l'ANSTJ puis par Planète Sciences. Ayant pour objet la promotion et l'organisation d'activités techniques et pédagogiques autour des domaines des sciences et de l'aérospatial, elle est amenée à participer au C'Space organisé par Planète Science ainsi qu'à des événements comme EuRoC et d'autres campagnes indépendantes.

L'équipe du projet sera donc épaulée tout au long de l'année par Air ESIEA qui elle-même bénéficie du soutien opérationnel de ses partenaires. Le plus important d'entre eux restant l'école d'ingénieur à laquelle elle est rattachée : l'ESIEA.

L'ESIEA fournit à Air ESIEA depuis sa création des locaux à titre gracieux ainsi qu'une dotation pécuniaire qu'elle laisse à sa charge de répartir entre ses nombreux projets. La philosophie globale reste de faire des fusées aussi peu chères que possible, sans jamais compromettre la sécurité du public ou de la mission.

PRÉSENTATION

Gunther.ino est la carte électronique des minifusées d'Air ESIEA. Elle détecte le décollage et l'apogée, commande l'éjection du parachute et enregistre les données du vol.

En mode classique, la carte se programme exactement comme une Arduino Nano : `pinMode`, `digitalWrite`, `analogRead`, `Wire`, `SPI`, `Servo`... Aucune bibliothèque spéciale n'est nécessaire : chaque broche a déjà un nom (`LED_1`, `JACK_1`, `SERVO_1`, `SD_CS`...). Ce manuel décrit ce qu'il y a derrière chaque nom et ce que chaque broche et chaque bus savent faire.

Caractéristiques

Élément	Détail
Microcontrôleur	ATSAMD51J20A : Cortex-M4F à 120 MHz, 1 Mo de flash, 256 Ko de RAM
Baromètre	BMP390 (I2C, adresse 0x76) : pression, température, altitude. BMP388 et SPL07-003 acceptés
Centrale inertielle	BMI088 (SPI) : accéléromètre ±24 g et gyroscope ±2000 °/s
Stockage	Lecteur de carte micro SD (SPI), avec détection de carte
Sorties	4 LED, 4 servos, 1 buzzer piézo
Entrées	4 switchs, 1 jack de décollage, mesure de la tension pile
Extensions	1 bus I2C, 1 bus SPI avec un CS libre, 1 UART
Alimentation	Pile 9 V (7 à 12 V) ou USB-C
Programmation	USB-C avec l'Arduino IDE (bootloader UF2), ou sonde SWD

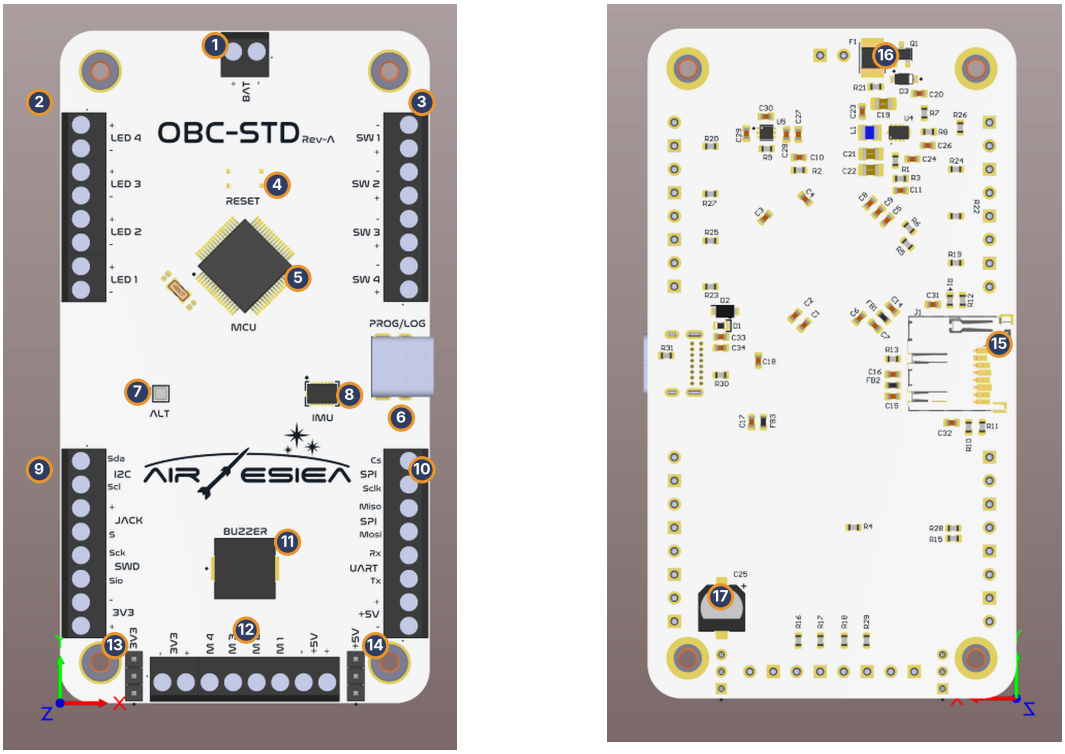
Deux façons de programmer la carte

	Mode simplifié	Mode classique
Principe	Bibliothèque `Gunther` : une fonction par action (`Gunther.altitude()`, `Gunther.apogee()`...)	Code Arduino standard, comme sur une Arduino Nano, avec les noms des broches de la carte
Pour qui	Premiers vols, équipes qui veulent un programme fiable rapidement	Ceux qui veulent écrire eux-mêmes la lecture des capteurs et la logique du vol
Exemples	**Fichier › Exemples › Gunther**	**Fichier › Exemples › Gunther Classique**
Manuel	Manuel d'utilisation, mode simplifié	Manuel d'utilisation, mode classique

Ce manuel couvre le mode **classique**. Les deux modes peuvent être mélangés dans le même programme : voir le manuel du mode simplifié pour l'autre partie.

VUE D'ENSEMBLE

Les numéros sur les images renvoient au tableau en dessous.



Dessus et dessous de la carte

N°	Élément	Rôle
1	BAT	Entrée pile (+ et −)
2	LED 1 à LED 4	Sorties LED
3	SW 1 à SW 4	Entrées switchs
4	RESET	Redémarrage (double appui = bootloader)
5	MCU	Microcontrôleur ATSAMD51J20A
6	PROG/LOG	USB-C : programmation et moniteur série
7	ALT	Baromètre BMP390
8	IMU	Centrale inertielle BMI088
9	Bornier gauche	I2C, JACK, SWD, 3V3
10	Bornier droit	SPI, UART, +5V
11	BUZZER	Buzzer piézo
12	Bornier du bas	3V3, servos M1 à M4, +5V
13	Barrette 3V3	Alimentation 3,3 V
14	Barrette +5V	Alimentation 5 V des servos
15	Carte SD	Lecteur micro SD (dessous)
16	Protection	Fusible réarmable 1,5 A et anti-inversion de la pile
17	Condensateur	Réserve d'énergie du +5V (220 µF)

ALIMENTATION

La pile arrive sur **BAT**, passe par la protection, puis un convertisseur fait le **5 V** et un régulateur fait le **3,3 V** à partir du 5 V. L'USB-C fournit aussi le 5 V, à travers une diode : les deux peuvent être branchés en même temps.

Rail	Où le trouver	Limite
BAT	Bornier BAT	7 à 12 V (pile 9 V). Fusible 1,5 A, protégé contre l'inversion
+5V	Bornier droit, bornier du bas, barrette +5V	2 A au total, partagé avec le 3,3 V
3V3	Bornier gauche, bornier du bas, barrette 3V3	500 mA au total, dont environ 50 mA pour la carte
USB-C	Prise PROG/LOG	500 mA (limite du port USB du PC)

La tension de la pile arrive sur la broche `VBAT_SENSE` (PB00, `A0`) à travers un pont diviseur 30k/10k (rapport 1/4, constante `VBAT_DIVIDER_RATIO`). `readBatteryVoltage()` donne directement la tension en volts.

> Les borniers +5V et 3V3 sont reliés directement aux alimentations de la carte : un court-circuit entre + et − coupe toute la carte.

INSTALLATION ET OUTILS

Installation du paquet Arduino

1. **Fichier › Préférences**, puis coller ce lien dans **URL de gestionnaire de cartes supplémentaires** :

```
https://raw.githubusercontent.com/Arkyos03/Gunther.ino-Arduino/main/package_gunther_index.json
```

2. **Outils › Type de carte › Gestionnaire de cartes** : chercher **Gunther.ino** et installer.
3. Choisir la carte **Gunther.ino (Rev A)** et le port USB.
4. Écrire le programme puis cliquer sur **Téléverser**, comme pour une carte Arduino.

Si le téléversement ne marche plus (programme planté), un **double appui sur RESET** met la carte en mode bootloader : la LED 1 pulse et un lecteur **GUNTHERBOOT** apparaît. Il suffit de relancer le téléversement.

Menu Outils

Menu	Choix	Usage
Optimisation	Petite taille (-Os, par défaut)	Programme le plus compact
	Rapide (-O2) · Plus rapide (-O3)	Calculs plus rapides, programme un peu plus gros
	Débogage (-Og -g3)	Pour le bouton Déboguer : les variables restent lisibles
Programmateur	CMSIS-DAP	Raspberry Pi Debug Probe, Atmel-ICE, MCU-Link, DAPLink…
	ST-Link V2	Original ou clone

Sonde SWD

Une sonde branchée sur le bornier SWD (Sck, Sio, GND sur la borne − du 3V3) permet trois choses :

- **Outils › Graver la séquence d'initialisation** : installe le bootloader sur une carte neuve.
- **Croquis › Téléverser avec un programmateur** : téléverse sans passer par l'USB.
- **Bouton Déboguer** (Arduino IDE 2) : points d'arrêt, pas à pas, variables et registres du SAMD51. Choisir **Outils › Optimisation › Débogage** avant.

> Avec un clone de ST-Link, la broche RST ne marche souvent pas : maintenir le bouton RESET de la carte et le relâcher au lancement d'OpenOCD.

LES BORNIERS

Pour chaque bornier : le repère écrit sur la carte, le nom à utiliser dans le code, la broche du microcontrôleur et ce qu'elle sait faire.

LED (bornier gauche, en haut)

Repère	Nom Arduino	Broche	Capacités
LED 1	`LED_1`	PB10	Sortie, PWM (`analogWrite`), interruption
LED 2	`LED_2`	PB11	Sortie, PWM (`analogWrite`), interruption
LED 3	`LED_3`	PB23	Sortie, interruption
LED 4	`LED_4`	PB22	Sortie, interruption

Borne + : sortie de la broche à travers une résistance de 330 Ω. Borne − : GND. La LED se branche directement, sans résistance.

Switchs (bornier droit, en haut)

Repère	Nom Arduino	Broche	Capacités
SW 1	`SWITCH_1` (`A1`)	PB04	Entrée, interruption, analogique
SW 2	`SWITCH_2` (`A2`)	PB05	Entrée, interruption, analogique
SW 3	`SWITCH_3` (`A3`)	PB06	Entrée, interruption, analogique
SW 4	`SWITCH_4` (`A4`)	PB07	Entrée, interruption, analogique

Borne + : broche à travers une résistance de 1 kΩ. Borne − : GND. Avec `pinMode(SWITCH_1, INPUT_PULLUP)` : switch ouvert = HIGH, fermé = LOW.

Jack de décollage (bornier gauche)

Repère	Nom Arduino	Broche	Capacités
JACK S	`JACK_1` (`A5`)	PB08	Entrée, interruption, analogique

Borne + : 3,3 V. Borne S : broche à travers 1 kΩ, avec un pull-down de 10 kΩ sur la carte. Jack en place = HIGH, jack arraché = LOW. `pinMode(JACK_1, INPUT_PULLDOWN)` conseillé.

Servos (bornier du bas)

Repère	Nom Arduino	Broche	Capacités
M1	`SERVO_1`	PA18	PWM 50 Hz (bibliothèque `Servo`), interruption
M2	`SERVO_2`	PA19	PWM 50 Hz (bibliothèque `Servo`), interruption
M3	`SERVO_3`	PA20	PWM 50 Hz (bibliothèque `Servo`), interruption
M4	`SERVO_4`	PA21	PWM 50 Hz (bibliothèque `Servo`), interruption

Signal à travers une résistance de 220 Ω. Les 4 sorties sont générées par le matériel (timer TCC1) : elles ne bloquent pas le programme. L'alimentation des servos se prend sur le +5V.

I2C (bornier gauche)

Repère	Nom Arduino	Broche	Capacités
Sda	`SDA`	PB02	Données I2C
Scl	`SCL`	PB03	Horloge I2C

Bus `Wire` (SERCOM5). Pull-ups de 4,7 kΩ déjà sur la carte. Le baromètre occupe l'adresse **0x76**.

SPI (bornier droit)

Repère	Nom Arduino	Broche	Capacités
Cs	`SPI1_CS_EXT` (ou `SPI_CS_EXT`)	PA11	Sortie (chip select), analogique, PWM
Sclk	`PIN_SPI1_SCK`	PB13	Horloge de `SPI1`
Miso	`PIN_SPI1_MISO`	PB14	Données entrantes de `SPI1`
Mosi	`PIN_SPI1_MOSI`	PB12	Données sortantes de `SPI1`

Bus `SPI1` (SERCOM4), partagé avec la centrale inertielle BMI088 : chaque composant a son propre CS.

UART (bornier droit)

Repère	Nom Arduino	Broche	Capacités
Tx	`PIN_SERIAL1_TX`	PA22	Émission de `Serial1`
Rx	`PIN_SERIAL1_RX`	PA23	Réception de `Serial1`

SWD (bornier gauche)

Repère	Broche	Rôle
Sck	PA30	SWCLK
Sio	PA31	SWDIO

COMPOSANTS DE LA CARTE

Composant	Bus / broche	Noms Arduino
Baromètre BMP390	I2C, adresse 0x76, INT sur PB15	`Wire`, `BMP390_INT`
Accéléromètre BMI088	SPI1, CS PA16, INT PB16	`SPI1`, `BMI088_ACCEL_CS`, `BMI088_ACCEL_INT`
Gyroscope BMI088	SPI1, CS PA17, INT PB17	`SPI1`, `BMI088_GYRO_CS`, `BMI088_GYRO_INT`
Carte micro SD	SPI (SERCOM2), CS PA15	`SPI`, `SD_CS`
Détection carte SD	PA27	`SD_DETECT` (ne fonctionne pas sur la Rev A)
Buzzer piézo	PB30	`BUZZER`
Mesure pile	PB00 (ADC)	`VBAT_SENSE` / `A0`
Port USB	USB natif	`Serial` (ou `SerialUSB`)

La carte SD et la centrale inertielle sont sur deux bus SPI séparés : écrire sur la carte SD ne ralentit pas la lecture de la centrale. Au démarrage, tous les CS (`SD_CS`, `BMI088_ACCEL_CS`, `BMI088_GYRO_CS`, `SPI1_CS_EXT`) sont mis à HIGH et les LED et le buzzer à LOW.

> Avec un **SPL07-003** à la place du BMP390 (même empreinte), la broche `BMP390_INT` n'est pas reliée et les registres sont différents : la bibliothèque `Gunther` le gère, en mode classique il faut son propre code.

CAPACITÉS DES BROCHES

Nom Arduino	N°	Broche	Numérique	PWM	Analogique	Interruption
`LED_1` · `LED_2`	0 · 1	PB10 · PB11	oui	oui	–	oui
`LED_3` · `LED_4`	2 · 3	PB23 · PB22	oui	–	–	oui
`SWITCH_1` à `_4`	4 à 7	PB04 à PB07	oui	–	oui	oui
`JACK_1`	8	PB08	oui	–	oui	oui
`SPI1_CS_EXT`	9	PA11	oui	oui	oui	–
`SERVO_1` à `_4`	10 à 13	PA18 à PA21	oui	50 Hz	–	oui
`BUZZER`	14	PB30	oui	oui	–	oui
`VBAT_SENSE` (`A0`)	15	PB00	–	–	oui	–
`BMP390_INT`	20	PB15	oui	–	–	oui
`BMI088_ACCEL_INT` · `_GYRO_INT`	26 · 27	PB16 · PB17	oui	–	–	oui
`SD_DETECT`	32	PA27	oui	–	–	oui

- **Numérique** : logique 3,3 V. `INPUT`, `OUTPUT`, `INPUT_PULLUP` et `INPUT_PULLDOWN`. `digitalToggle(broche)` inverse une sortie.
- **Interruption** : deux broches qui partagent la même ligne interne ne peuvent pas avoir une interruption en même temps. Les switchs, le jack, `BMP390_INT`, les deux `BMI088_*_INT` et `SD_DETECT` sont tous sur des lignes différentes et peuvent être utilisés ensemble.
- Les broches 35 à 45 (`PIN_EXT_PA02` … `PIN_EXT_PB31`), sauf la 40 (LED RGB), sont libres sur le microcontrôleur mais pas reliées à un bornier sur la Rev A.

ENTRÉES, SORTIES ET INTERRUPTIONS

```
volatile uint32_t tDecollage = 0;

void jackArrache() {              // jack arraché
  if (tDecollage == 0) tDecollage = millis();
}

void setup() {
  pinMode(LED_1, OUTPUT);
  pinMode(SWITCH_1, INPUT_PULLUP);
  pinMode(JACK_1, INPUT_PULLDOWN);
  attachInterrupt(digitalPinToInterrupt(JACK_1), jackArrache, FALLING);
}

void loop() {
  digitalWrite(LED_1, digitalRead(SWITCH_1) == LOW);   // switch fermé = LED
}
```

Fonction	Détail
`attachInterrupt(digitalPinToInterrupt(b), f, mode)`	Modes `RISING`, `FALLING`, `CHANGE`, `LOW`, `HIGH`. Filtre anti-parasite matériel activé
`detachInterrupt(digitalPinToInterrupt(b))`	Arrête l'interruption
`noInterrupts()` · `interrupts()`	Coupe / rétablit toutes les interruptions
`pulseIn(b, HIGH, timeout)`	Durée d'une impulsion en µs (récepteur RC, capteur à ultrasons…)
`shiftOut` · `shiftIn`	Registre à décalage

ENTRÉES ET SORTIES ANALOGIQUES

analogRead

0 à 3,3 V, **10 bits par défaut** (0 à 1023, comme une Nano). `analogReadResolution(12)` donne 0 à 4095, la résolution réelle du convertisseur.

analogReference(...)	Pleine échelle
`AR_DEFAULT`	3,3 V (par défaut)
`AR_INTERNAL1V0` · `1V1` · `1V2` · `1V25`	1,0 · 1,1 · 1,2 · 1,25 V (référence interne)
`AR_INTERNAL2V0` · `2V2` · `2V4` · `2V5` (`AR_INTERNAL`)	2,0 · 2,2 · 2,4 · 2,5 V (référence interne)
`AR_INTERNAL1V65`	1,65 V (3,3 V / 2)
`AR_EXTERNAL`	Tension sur la broche VREFA (PA03)

```
analogReadResolution(12);
int brut = analogRead(VBAT_SENSE);
float vPile = brut * 3.3 / 4095.0 * VBAT_DIVIDER_RATIO;
```

analogWrite

- **PWM** à 732 Hz sur `LED_1`, `LED_2`, `BUZZER` et `SPI1_CS_EXT` ; 8 bits par défaut (0 à 255), jusqu'à 16 bits avec `analogWriteResolution`.
- Sur `SERVO_1` à `SERVO_4`, la PWM est à 50 Hz : utiliser plutôt la bibliothèque `Servo`.
- **Vraie tension** (convertisseur numérique-analogique, 12 bits) sur `PIN_EXT_PA02` et `PIN_EXT_PA05`, broches libres non reliées à un bornier sur la Rev A.
- Sur une broche sans PWM, `analogWrite` met la sortie à HIGH au-dessus de la moitié, LOW en dessous.

TEMPS ET SON

Fonction	Détail
`millis()` · `micros()`	Temps depuis le démarrage en ms / µs
`delay(ms)` · `delayMicroseconds(us)`	Attente. L'USB continue de fonctionner pendant `delay`
`tone(BUZZER, f)` · `tone(BUZZER, f, durée)`	Son de fréquence f (Hz), sans bloquer le programme
`noTone(BUZZER)`	Arrête le son

`tone` fonctionne sur n'importe quelle broche, une seule à la fois.

LIAISONS SÉRIE

Port	Broches	Détail
`Serial` (`SerialUSB`)	USB-C	Moniteur et traceur série. La vitesse de `Serial.begin()` est ignorée (USB)
`Serial1`	Tx PA22, Rx PA23	UART 3,3 V. `Serial1.begin(9600)` ou `Serial1.begin(9600, SERIAL_8E1)`

- Formats de `Serial1` : `SERIAL_8N1` (défaut), `SERIAL_8N2`, `SERIAL_8E1`, `SERIAL_8E2`, `SERIAL_8O1`, `SERIAL_8O2`, `SERIAL_7N1`, `SERIAL_7E1`, `SERIAL_7O1`.
- `Serial.printf("%d %.2f\n", a, b)` est disponible sur tous les ports (nombres à virgule compris).
- `while (!Serial && millis() < 3000) {}` attend l'ouverture du moniteur série au plus 3 s.

BUS I2C (WIRE)

`#include <Wire.h>`. Même utilisation que sur toutes les cartes Arduino.

Fonction	Détail
`Wire.begin()`	Mode maître
`Wire.setClock(400000)`	100 kHz par défaut, de 10 kHz à 1 MHz
`Wire.beginTransmission(adr)` · `write` · `endTransmission()`	Écriture. Retour : 0 OK, 2 adresse sans réponse, 3 donnée refusée, 4 erreur, 5 délai dépassé
`Wire.requestFrom(adr, n)` · `read()`	Lecture de n octets (256 maximum)
`Wire.setWireTimeout(us)`	Délai max d'une transaction (25 ms par défaut). Un bus bloqué est débloqué tout seul
`Wire.begin(adr)` · `onReceive(f)` · `onRequest(g)`	Mode esclave : la carte répond à un autre microcontrôleur

> En mode esclave, la carte ne peut plus lire son propre baromètre (il est sur le même bus).

Baromètre BMP390 / BMP388 en direct

Registre	Rôle
0x00	CHIP_ID : 0x60 (BMP390) ou 0x50 (BMP388 / BMP384)
0x04 à 0x09	Pression (3 octets) puis température (3 octets), poids faible en premier
0x1B · 0x1C · 0x1D · 0x1F	Mode · suréchantillonnage · fréquence · filtre
0x31 à 0x45	Coefficients de calibration (21 octets)
0x7E	Commande (0xB6 = reset)

Le calcul complet de la pression à partir des coefficients est dans l'exemple **07_Barometre_I2C**. La bibliothèque **Adafruit BMP3XX** du gestionnaire de bibliothèques fonctionne aussi.

BUS SPI

Bus	Broches (MOSI, SCK, MISO)	Composants
SPI	PA12, PA13, PA14	Carte micro SD (SD_CS)
SPI1	PB12, PB13, PB14	BMI088 (BMI088_ACCEL_CS, BMI088_GYRO_CS), capteur externe (SPI1_CS_EXT)

```
SPI1.begin();
SPI1.beginTransaction(SPISettings(8000000, MSBFIRST, SPI_MODE0));
digitalWrite(SPI1_CS_EXT, LOW);
uint8_t r = SPI1.transfer(0x80 | registre);
uint8_t v = SPI1.transfer(0);
digitalWrite(SPI1_CS_EXT, HIGH);
SPI1.endTransaction();
```

- Jusqu'à 24 MHz, modes `SPI_MODE0` à `SPI_MODE3`. Transferts de blocs avec `transfer(buffer, n)`.
- **BMI088, accéléromètre** : une première lecture le fait passer en SPI, et chaque lecture commence par un octet vide à ignorer. CHIP_ID = 0x1E (registre 0x00), données en 0x12.
- **BMI088, gyroscope** : lecture normale. CHIP_ID = 0x0F, données en 0x02.
- Exemple complet : **08_Centrale_SPI1**. La bibliothèque **Bolder Flight BMI088** fonctionne aussi.

CARTE SD

La bibliothèque **SdFat** est fournie avec la carte. Cartes FAT16, FAT32 ou exFAT.

```
#include <SdFat.h>
SdFat sd;
FsFile f;


sd.begin(SdSpiConfig(SD_CS, DEDICATED_SPI, SD_SCK_MHZ(12), &SPI));
f.open("vol.csv", O_WRONLY | O_CREAT | O_APPEND);
f.println("temps;altitude");
f.sync();                          // écrit vraiment sur la carte
```

- Sur la Rev A, `SD_DETECT` reste toujours à HIGH (contact de détection non câblé à la masse) : une carte est présente quand `sd.begin()` réussit.
- Appeler `sync()` régulièrement (toutes les 0,5 s par exemple) : en cas de coupure, seules les données écrites depuis le dernier `sync()` sont perdues.

SERVOS

```
#include <Servo.h>
Servo trappe;


trappe.attach(SERVO_1);            // ou attach(SERVO_1, 1000, 2000)
trappe.write(90);                  // angle en degrés
trappe.writeMicroseconds(1500);    // ou impulsion en µs
trappe.detach();                   // coupe le signal
```

4 servos au maximum, uniquement sur SERVO_1 à SERVO_4.

EEPROM ET WATCHDOG

EEPROM

`#include <EEPROM.h>`. 4096 octets gardés après une coupure et après un nouveau téléversement (ils sont dans une zone réservée de la mémoire flash). Même utilisation que sur une Arduino Uno.

Fonction	Détail
`EEPROM.read(i)` · `EEPROM.write(i, v)` · `EEPROM.update(i, v)`	Un octet (255 si jamais écrit)
`EEPROM.get(i, var)` · `EEPROM.put(i, var)`	Une variable ou une structure entière
`EEPROM[i]`	S'utilise comme un tableau
`EEPROM.setCommitASAP(false)` · `EEPROM.commit()`	Grouper plusieurs écritures en une seule
`EEPROM.clear()`	Tout remettre à 255

> Chaque écriture efface et réécrit un bloc de flash, qui supporte environ 100 000 effacements. Ne jamais écrire dans l'EEPROM à chaque tour de `loop()` : les données de vol vont sur la carte SD.

Watchdog

`#include <Watchdog.h>`. Si le programme se bloque plus longtemps que le délai choisi, la carte redémarre.

Fonction	Détail
`Watchdog.enable(ms)`	Active (8 ms à 16 s). Renvoie le délai réel en ms
`Watchdog.reset()`	À appeler plus souvent que le délai
`Watchdog.disable()`	Désactive
`Watchdog.resetCause()` · `resetCauseText()`	Cause du dernier redémarrage : mise sous tension, tension trop basse, bouton RESET, watchdog, logiciel
`Watchdog.restart()`	Redémarre immédiatement

FONCTIONS PROPRES À LA CARTE

Fonction	Détail
`readBatteryVoltage()`	Tension de la pile en volts (moyenne de 8 mesures)
`digitalToggle(b)`	Inverse une sortie
`resetToBootloader()`	Redémarre en mode bootloader (comme un double appui sur RESET)
`mapFloat(x, a, b, c, d)`	Comme `map`, avec des nombres à virgule
`Serial.printf(...)`	Affichage formaté sur tous les ports série
`INPUT_PULLDOWN`	Résistance de rappel interne vers GND

EXEMPLES FOURNIS

Dans **Fichier › Exemples › Gunther Classique** :

Exemple	Contenu
01_Clignotant	`digitalWrite` sur une LED
02_LED_Variation	`analogWrite` sur LED 1 et LED 2
03_Switchs_Jack	`digitalRead` et interruption sur le jack
04_Buzzer	`tone` et `noTone`
05_Servo	Bibliothèque `Servo`
06_Batterie_ADC	`analogRead`, résolution et référence
07_Barometre_I2C	BMP390 / BMP388 lu en direct avec `Wire`, calcul de l'altitude
08_Centrale_SPI1	BMI088 lu en direct avec `SPI1`
09_Carte_SD	Fichier CSV avec SdFat
10_UART_Serial1	Passerelle entre l'USB et `Serial1`
11_EEPROM	Compteur de démarrages et réglages gardés en mémoire
12_Watchdog	Redémarrage automatique si le programme se bloque
13_I2C_Esclave	La carte commandée par un autre microcontrôleur
14_Vol_Classique	Vol complet sans la bibliothèque Gunther

CONTACT ET VERSION

Références

- Code et paquet Arduino : https://github.com/Arkyos03/Gunther.ino-Arduino
- Pour plus d'information sur Air ESIEA : https://airesiea.org

Contact

Pour toute question sur la carte, contactez Arkyos03 ou l'association :

Courriel : air-esiea@et.esiea.fr

Date et version

Manuel mode classique : version 1.2, 26/10/2026. Carte : Gunther.ino Rev A. Paquet Arduino : 0.1.5.